

# 智慧银行实验教程

## 实验一：AI 辅助金融科技开发实践

姓名：龙玲玲 学号：202301775 班级：金融 202302

实验日期：2026 年 6 月 25 日

实验环境：Traz IDE + AI 智能体 + CNB 平台

## 实验目的

本次实验旨在通过 AI 智能体协作，完成以下学习目标：

- 掌握 AI 辅助编程的基本流程与提示词编写技巧
- 使用 Trae IDE 进行前端项目开发（贪吃蛇游戏）
- 运用 AI 进行学术论文解读与文献分析
- 完成 Excel 数据合并与信用卡消费预测分析
- 使用 CNB 平台进行代码版本管理与协作推送
- 理解 BMAD 方法论在金融科技项目中的实践应用

## 实验环境

| 环境项  | 配置信息                                   |
|------|----------------------------------------|
| 操作系统 | macOS (Anaconda Navigator 环境)          |
| IDE  | Trae IDE (AI 智能体集成)                    |
| 前端技术 | HTML5 + CSS3 + JavaScript (Canvas API) |
| 数据分析 | Python 3 + Pandas + NumPy + Matplotlib |
| 预测模型 | SARIMA + LSTM (PyTorch)                |
| 文档工具 | pandoc + LibreOffice                   |
| 协作平台 | CNB (cnb.cool)                         |
| 浏览器  | Chrome (Chrome DevTools MCP)           |

## 实验步骤

### Trae IDE 安装与配置

首先安装 Trae IDE 并完成 AI 智能体配置。安装完成后，IDE 界面如下：



图 1 Trae IDE 安装界面

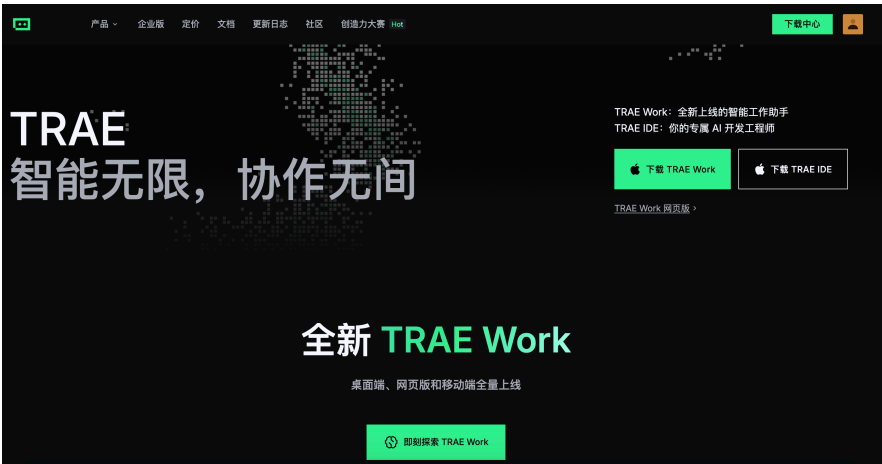


图 2 Trae IDE 配置界面

贪吃蛇游戏开发

使用 AI 智能体生成纯 HTML+CSS+原生 JavaScript 贪吃蛇游戏，运行效果如下：



图 3 贪吃蛇游戏运行截图

提示词自评：该提示词结构清晰、需求明确，技术栈限定为纯 HTML+CSS+原生JS，画布尺寸和颜色也给了具体数值，核心玩法要素齐全。但仍缺少帧率（如 150ms 一帧）、单元格大小（如 20px）、蛇的初始长度和位置、分数显示位置等细节。

## 学术论文解读

设计智能体提示词，用于解读银行业/金融科技领域学术论文。提示词要求按以下结构输出：论文基本信息、研究问题与背景、方法论、核心发现、术语速查表、可复现性评估。

选取 5 篇相关文献进行解读，涵盖信用风险评估、LSTM 信用预测、SARIMA-GARCH 模型、深度学习欺诈检测、XGBoost 客户流失预测等方向。

## Excel 数据合并与预测分析

将 5 个 CSV 文件合并为统一 Excel 文件，基于 customer\_id + transaction\_date 去重，合并前 302 行，去重后 251 行。随后进行信用卡消费预测分析，对比 SARIMA 和 LSTM 两种模型。

## CNB 平台代码推送

将实验代码和数据同步至 CNB 平台仓库：

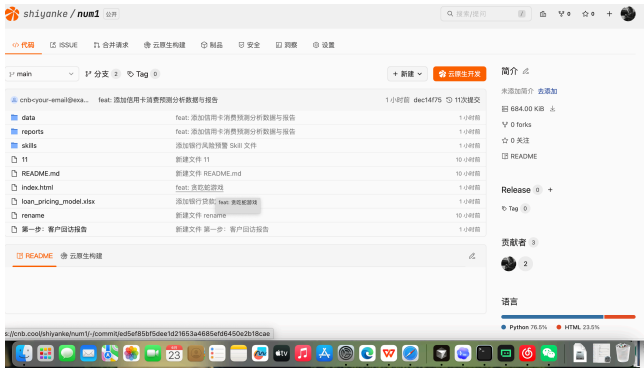


图 4 CNB 平台推送截图

仓库地址：<https://cnb.cool/shiyanke/num1>

## 实验结果

### 贪吃蛇游戏

成功生成可运行的贪吃蛇游戏，包含画布渲染、键盘控制、碰撞检测、分数统计等核心功能。

### 论文解读成果

完成 5 篇金融科技领域论文的结构化解读，核心发现包括：

- LSTM 能捕捉时间序列中的长期依赖关系，在信用评分中表现优异
- SMOTE 过采样能有效提升不平衡数据中少数类的识别率
- 深度学习模型对特征工程依赖较低，适合处理高维金融数据
- 模型可解释性在实际应用中仍是重要考量因素

### 数据合并与预测结果

| 指标        | 数值                      |
|-----------|-------------------------|
| 输入文件数     | 5 个 (.csv)              |
| 合并前总行数    | 302 行                   |
| 去重后总行数    | 251 行                   |
| 移除重复行数    | 51 行                    |
| 数据时间范围    | 2022-01-01 至 2024-12-31 |
| 原始缺失值     | 52 个 (4.7%)             |
| IQR 异常值标注 | 54 个 (4.9%)             |
| 节日消费提升    | +74.5%                  |

| 模型                     | RMSE   | MAPE   |
|------------------------|--------|--------|
| SARIMA(1,1,1)(1,1,1,7) | 1.8388 | 25.76% |
| LSTM(窗口=14, 隐层=64)     | 1.8254 | 25.33% |

## 问题与解决

---

### 变量作用域错误导致 NameError

问题：在编写 `forecast_analysis.py` 时，将 LSTM 参数（WINDOW、HIDDEN、EPOCHS、LR）定义在 `if HAS_TORCH:` 条件判断内部。由于初始环境未安装 PyTorch，这些变量未被定义，后续报告模板引用时抛出 `NameError`。

解决：① 将参数定义移到条件判断之前，设置默认值；② 将统计量计算移到条件判断外面；③ 确保报告模板中所有变量都有默认值。

### CNB 平台同步权限不足

问题：多次尝试将代码同步到 CNB 平台均失败，报错信息不明确。

解决：经 AI 分析，发现默认生成的 token 未开启写入权限。重新配置写入权限后同步成功。

### 数据列名不一致

问题：Excel 合并任务中，各 CSV 文件列名不统一，存在大量缺失值。

解决：通过规范化列名映射和缺失值填补完成数据整合。

## 实验心得

---

在本次实验中，我通过与 AI 智能体协作完成了多项金融科技相关任务，包括贪吃蛇游戏开发、Excel 数据合并、信用卡消费预测分析等。整个过程让我深刻体会到 AI 在编程和数据分析领域的强大能力，同时也发现了一些需要注意的问题。

首先，变量作用域管理是条件分支代码中的关键。在 `forecast_analysis.py` 中，由于参数定义位置不当导致 `NameError`，这让我意识到写带有条件分支的代码时，必须逐个检查变量的作用域，不能让模板和输出部分引用可能不存在的变量。加异常处理和默认值是提升代码健壮性的好习惯。

其次，平台权限配置需要仔细核对。CNB 同步失败的经历提醒我，遇到报错时把错误信息交给 AI 分析往往能快速找到突破口。同时配置平台权限时要逐项检查，不能想当然地认为默认配置就是对的。

第三，数据预处理的重要性不容忽视。Excel 合并任务中列名不一致、缺失值众多，数据质量差得无法直接使用。通过规范化列名映射和缺失值填补才完成整合。不管做什么分析，数据清洗和预处理都是跳不过去的第一步。

第四，模型选择需结合业务需求。SARIMA 适合周期性明显的数据，LSTM 能捕捉更复杂的非线性模式。最终选择取决于业务需求和数据特点，不能一概而论。

第五，版本控制的价值在实践中得以体现。用 Git 管理代码变更不仅方便团队协作，还能保证代码的安全性和可追溯性。

最后，AI 的文档生成能力超出预期。它不仅能写代码，还能输出结构化的报告文档，让我能把更多精力放在分析和决策上。AI 智能体是强大的工具，但使用时不能掉以轻心，需要仔细核对输出，将 AI 的优势与人的判断力、创造力结合起来，才能高效完成复杂任务。